Firm Admin Onboarding Flow

Complete guide: how a Super Admin creates firms, customizes permissions, and onboard Firm Admins with different access levels.

1. The Building Blocks

Concept	Description	Example
Permission	Smallest unit. One action on one feature. Has a scope (GLOBAL / TENANT / ASSIGNED / OWN).	`AUDIT:ACCESS` (scope: TENANT) `USER_MANAGEMENT:VIEW` (scope: TENANT)
Module	Groups permissions under one heading in the frontend permission picker.	Module "AUDIT" contains: ACCESS, VIEW, CREATE, EDIT, DELETE
Role	A named collection of permissions. A user gets ALL permissions in their role.	FIRM_ADMIN has 48 permissions across 8 modules
User	Has one role. Inherits all permissions from that role.	Rajesh (FIRM_ADMIN) can see everything FIRM_ADMIN role allows

Permission Scopes

Scope	Who Can Receive
GLOBAL	SUPER_ADMIN only
TENANT	FIRM_ADMIN and below (most module permissions)
ASSIGNED	ADVOCATE / PARALEGAL and below
OWN	CLIENT only

> **Key rule:** When you create a permission via `POST /api/v1/admin/permissions`, set `"scope": "TENANT"` unless it is SUPER_ADMIN-only. New permissions are auto-assigned to the SUPER_ADMIN system role. FIRM_ADMIN must be manually assigned by the super admin.

2. System Roles (Pre-seeded on Startup)

Role Code	Scope Ceiling	Default Permissions
SUPER_ADMIN	GLOBAL	All modules
FIRM_ADMIN	TENANT	Case Mgmt, Employee, Client, Billing, Documents, Calendar, Reports, Audit
ADVOCATE	ASSIGNED	Case view/edit, documents, calendar
PARALEGAL	ASSIGNED	Case view, documents, calendar view
CLIENT	OWN	Own case view, own documents

3. Complete Onboarding Flow

Scenario: Super Admin needs to onboard two firms with different permission sets.
Firm A (Apex Law) — Firm Admin needs ONLY the Audit module.
Firm B (Summit Legal) — Firm Admin needs Audit + User Management.

Step 1 Super Admin creates Firm A

```
POST /api/v1/super-admin/firms
{
  "firmName": "Apex Law Associates",
  "firmCode": "CLAW",
  "firmType": "LAW_FIRM",
  "adminName": "Rajesh Hamal",
  "adminEmail": "rajesh@apexlaw.com",
  "adminUsername": "rajesh_admin",
  "adminPassword": "Temp@123"
}
```

What happens internally:

1. Firm record created with code `CLAW`
2. 4 system roles cloned for this firm: FIRM_ADMIN, ADVOCATE, PARALEGAL, CLIENT
3. Each cloned role gets ALL permissions from the system template
4. Rajesh is created as FIRM_USER with the cloned FIRM_ADMIN role
5. Welcome email is sent to rajesh@apexlaw.com

At this point: Rajesh has ALL standard permissions (8 modules).

Step 2 Super Admin creates Firm B

```
POST /api/v1/super-admin/firms
{
  "firmName": "Summit Legal",
  "firmCode": "SUMMIT",
  "firmType": "LAW_FIRM",
  "adminName": "Anita Sharma",
  "adminEmail": "anita@summitlegal.com",
  "adminUsername": "anita_admin",
  "adminPassword": "Temp@123"
}
```

Same internal process: Anita gets the standard FIRM_ADMIN role with all 8 modules.

↓

Step 3 Super Admin finds Firm A's cloned FIRM_ADMIN role ID

```
GET /api/v1/admin/roles
```

Look for the role where `"code": "FIRM_ADMIN"` and `"isSystem": false`.
The `id` field is the cloned role ID for Firm A.

Example response extract:

```
{
  "code": "FIRM_ADMIN",
  "id": "bcd8721f-0a06-4fd0-9d70-f3070b9513bf",
  "isSystem": false,
  ...
}
```

↓

Step 4 Super Admin restricts Firm A to only Audit permissions

```
POST /api/v1/admin/roles
{
  "data": {
    "id": "bcd8721f-0a06-4fd0-9d70-f3070b9513bf",
```

```
    "name": "FIRM_ADMIN",
    "code": "FIRM_ADMIN",
    "permissionIds": [
      "64db31da-aed1-4cba-9ebf-72666a2d9fc4",   // AUDIT:ACCESS
      "31253422-2de3-4682-99c0-3127db1f286f",   // AUDIT:VIEW
      "7370d07e-4a7c-48d1-98f8-9cd1af6b1c36",   // AUDIT:CREATE
      "3296d115-70e6-47c4-910e-7f123b14ccf4",   // AUDIT:EDIT
      "6c5beaa7-ea7d-4ca9-a364-7f8a0ccf1c26"    // AUDIT:DELETE
    ]
  }
}
```

Result: Rajesh can now ONLY see the Audit module in his sidebar. Case Management, Employee, Billing, etc. are hidden. His `/me` response will only contain AUDIT permissions.

↓

Step 5 Super Admin finds Firm B's cloned FIRM_ADMIN role ID

```
GET /api/v1/admin/roles
```

Same process — find the FIRM_ADMIN role where `"code": "FIRM_ADMIN"` and `"isSystem": false`, belonging to Firm B (`SUMMIT`).

↓

Step 6 Super Admin sets Firm B to Audit + User Management

```
POST /api/v1/admin/roles
{
  "data": {
    "id": "<firm-b-role-id>",
    "name": "FIRM_ADMIN",
    "code": "FIRM_ADMIN",
    "permissionIds": [
      // Audit permissions
      "64db31da-aed1-4cba-9ebf-72666a2d9fc4",   // AUDIT:ACCESS
      "31253422-2de3-4682-99c0-3127db1f286f",   // AUDIT:VIEW
      "7370d07e-4a7c-48d1-98f8-9cd1af6b1c36",   // AUDIT:CREATE
      "3296d115-70e6-47c4-910e-7f123b14ccf4",   // AUDIT:EDIT
      "6c5beaa7-ea7d-4ca9-a364-7f8a0ccf1c26",   // AUDIT:DELETE
      // User Management permission
      "eb4b549c-3300-4c99-9d93-f0c614a93b65"    // USER_MANAGEMENT:VIEW
    ]
```

```
  }
}
```

Result: Anita can see both Audit AND User Management modules. Her `/me` response will contain both sets of permissions.

↓

Step 7 Frontend reads /me for each admin

Firm A admin (Rajesh) logs in:

```
GET /api/v1/me
Response:
{
  "permissions": ["AUDIT:ACCESS", "AUDIT:VIEW", "AUDIT:CREATE", "AUDIT:EDIT", "AUDIT:DELETE"],
  "permissionGroups": {
    "AUDIT": ["ACCESS", "VIEW", "CREATE", "EDIT", "DELETE"]
  }
}
```

Frontend sidebar: Only "Audit Logs" module visible.

Firm B admin (Anita) logs in:

```
GET /api/v1/me
Response:
{
  "permissions": [
    "AUDIT:ACCESS", "AUDIT:VIEW",
    "USER_MANAGEMENT:VIEW"
  ],
  "permissionGroups": {
    "AUDIT": ["ACCESS", "VIEW"],
    "USER_MANAGEMENT": ["VIEW"]
  }
}
```

Frontend sidebar: "Audit Logs" + "User Management" modules visible.

4. Alternative: Custom Roles for Reusable Permission Sets

If you frequently need the same restricted permission set (e.g., "Audit Only"), create a **custom role** instead of hand-picking IDs each time.

Create a custom role

```
POST /api/v1/admin/roles
{
  "data": {
    "name": "AUDIT_ONLY_ADMIN",
    "code": "AUDIT_ONLY_ADMIN",
    "description": "Firm admin with audit access only",
    "permissionIds": [
      "64db31da-aed1-4cba-9ebf-72666a2d9fc4",   // AUDIT:ACCESS
      "31253422-2de3-4682-99c0-3127db1f286f",   // AUDIT:VIEW
      "7370d07e-4a7c-48d1-98f8-9cd1af6b1c36",   // AUDIT:CREATE
      "3296d115-70e6-47c4-910e-7f123b14ccf4",   // AUDIT:EDIT
      "6c5beaa7-ea7d-4ca9-a364-7f8a0ccf1c26"    // AUDIT:DELETE
    ]
  }
}
```

Assign the custom role to an employee

```
PATCH /api/v1/firm/employees/{employeeId}/role
{
  "data": {
    "roleId": "<custom-role-id>"
  }
}
```

> **Note:** Custom roles are system-level (not per-firm). They are available to all firms. Create them with clear naming conventions like `FIRM_RESTRICTED_AUDIT`.

5. Frontend Permission Gating Logic

On every app load, call `GET /api/v1/me` and store the permissions array in application state:

```
// On app load
const me = await fetch('/api/v1/me').then(r => r.json());
const perms = me.data.permissions;        // flat array
const groups = me.data.permissionGroups;  // grouped by module

// Helper function used everywhere in the UI
function hasPermission(code) {
  return perms.includes(code);
}

// Sidebar rendering
const sidebarModules = [
  { name: "Case Management",  perm: "CASE_MANAGEMENT:ACCESS" },
  { name: "Employee",         perm: "EMPLOYEE:ACCESS" },
```

```
  { name: "Client Management", perm: "CLIENT_MANAGEMENT:ACCESS" },
  { name: "Billing",          perm: "BILLING:ACCESS" },
  { name: "Documents",        perm: "DOCUMENT_MANAGEMENT:ACCESS" },
  { name: "Calendar",         perm: "CALENDAR:ACCESS" },
  { name: "Reports",          perm: "REPORTS:ACCESS" },
  { name: "Audit Logs",       perm: "AUDIT:ACCESS" },
  { name: "User Management",  perm: "USER_MANAGEMENT:ACCESS" },
];

sidebarModules.forEach(module => {
  if (hasPermission(module.perm)) {
    renderSidebarItem(module.name);
  }
});

// Button-level gating
{hasPermission("CASE_MANAGEMENT:CREATE") && <CreateCaseButton />}
{hasPermission("EMPLOYEE:CREATE") && <AddEmployeeButton />}
```

6. API Reference Summary

Method	Endpoint	Who	Purpose
POST	/api/v1/super-admin/firms	SA	Create a firm + clone roles + create firm admin
GET	/api/v1/admin/roles	SA	List all roles (find the cloned FIRM_ADMIN IDs)
POST	/api/v1/admin/roles	SA	Update role name + permissions (single call)
GET	/api/v1/admin/permissions	SA	List all permissions (build the ID list for role update)
POST	/api/v1/admin/permissions	SA	Create a new permission (auto-assigned to SUPER_ADMIN)
POST	/api/v1/admin/modules	SA	Create a new module (for UI grouping)
PATCH	/api/v1/firm/employees/{id}/role	FA	Change an employee's role
GET	/api/v1/me	All	Get current user's identity + permissions